W3 Data cleaning - Si Jie, Lin
W4 EDA - Adrian, mc
W5Pre processing - Manu, Jeeva

- title: title name given to the earthquake (object)
- magnitude: The magnitude of the earthquake (float)
- date_time: date and time
- cdi: The maximum reported intensity for the event range
- mmi: The maximum estimated instrumental intensity for the event
- alert: The alert level - “green”, “yellow”, “orange”, and “red”
- tsunami: "1" for events in oceanic regions and "0" otherwise
- sig: A number describing how significant the event is. Larger numbers indicate a more significant event. This value is determined on a number of factors, including: magnitude, maximum MMI, felt reports, and estimated impact
- net: The ID of a data contributor. Identifies the network considered to be the preferred source of information for this event.
- nst: The total number of seismic stations used to determine earthquake location.
- dmin: Horizontal distance from the epicenter to the nearest station
- gap: The largest azimuthal gap between azimuthally adjacent stations (in degrees). In general, the smaller this number, the more reliable is the calculated horizontal position of the earthquake. Earthquake locations in which the azimuthal gap exceeds 180 degrees typically have large location and depth uncertainties
- magType: The method or algorithm used to calculate the preferred magnitude for the event
- depth: The depth where the earthquake begins to rupture
- latitude / longitude: coordinate system by means of which the position or location of any place on Earth's surface can be determined and described
- location: location within the country
- continent: continent of the earthquake hit country
- country: affected country

Objectives:

1. *"Identify and map the top 5 global seismic 'hot zones' by applying spatial clustering algorithms (e.g., DBSCAN or K-Means) on historical earthquakes*
2. *Develop and tune ensemble machine learning models (e.g., Random Forest, XGBoost) to predict tsunami occurrence (binary classification).*

Week 1: Data Preparation & Exploratory Data Analysis

The goal of this week is to clean the dataset and understand the spatial and temporal distributions of the earthquakes before applying algorithms.

- **Days 1-2: Data Cleaning & Formatting**
  - Load the dataset and handle missing values (e.g., dropping columns with too much missing data like continent, or imputing where appropriate).
  - Extract meaningful temporal features from date_time (Year, Month).
  - Filter and prep data (e.g., standardizing magnitude and depth).
- **Days 3-5: Geographic & Tsunami EDA**
  - Visualize global earthquake distributions using basic scatter maps to get an intuitive sense of the data.
  - Analyze the tsunami target variable to quantify the exact class imbalance (e.g., what percentage of earthquakes actually triggered a tsunami?).
  - Analyze feature distributions across the two classes (e.g., do tsunamis happen more often at specific depths or magnitudes?).
- **Days 6-7: Preprocessing for Clustering**
  - Isolate the features needed for Objective 1 (latitude, longitude, and potentially magnitude).
  - Scale the geographic coordinates using StandardScaler or geographic distance metrics (like Haversine distance) to prepare for K-Means/DBSCAN.

Week 2: Objective 1 — Spatial Clustering & "Hot Zone" Mapping

The goal of this week is to programmatically identify the top 5 most dangerous earthquake zones in the world.

- **Days 8-10: Applying Clustering Algorithms**
  - **K-Means:** Test K-Means clustering with the "Elbow Method" or Silhouette Scores to find an optimal number of global clusters.
  - **DBSCAN:** Apply DBSCAN, tuning the eps (radius) and min_samples parameters. DBSCAN is often better for geographical data because it identifies dense regions of arbitrary shapes (like fault lines) and ignores random outliers.
- **Days 11-12: Cluster Evaluation & Hot Zone Selection**
  - Compare the results of K-Means and DBSCAN. Select the algorithm that maps most closely to real-world fault lines.

    - Calculate the "danger score" of each cluster (e.g., by finding the clusters with the highest average magnitude or highest frequency of events).
    - Filter down to the **Top 5** clusters.
- **Days 13-14: Mapping Deliverable**
    - Use mapping libraries (like Folium, Plotly, or Geopandas) to plot these Top 5 clusters on an interactive world map.
    - **Deliverable 1 Complete:** A visual, algorithmic identification of global seismic hot zones.

Week 3: Objective 2 — Ensemble Modeling for Tsunami Prediction

The goal of this week is to build, train, and tune tree-based models to classify whether an earthquake will trigger a tsunami.

- **Days 15-16: Model Preprocessing & SMOTE**
    - Encode categorical features (like `magType` or `net`) using One-Hot Encoding or Label Encoding.
    - Split the data into Train and Test sets (80/20).
    - Apply **SMOTE** to the training set only, ensuring the model has enough synthetic "tsunami=1" examples to learn from without overfitting.
- **Days 17-19: Training Baseline & Ensemble Models**
    - Train a baseline Logistic Regression model just to set a performance benchmark.
    - Train your target ensemble models: **Random Forest Classifier** and **XGBoost Classifier**.
- **Days 20-21: Hyperparameter Tuning**
    - Use `GridSearchCV` or `RandomizedSearchCV` to optimize your models (e.g., tuning `max_depth`, `n_estimators`, and `learning_rate` for XGBoost).
    - Focus your optimization metric on **F1-Score** or **Recall**, since correctly identifying a tsunami (minimizing false negatives) is the most critical real-world outcome.

Week 4: Model Evaluation, Interpretability, & Reporting

The final week is about proving your model works, explaining how it makes decisions, and presenting your findings.

- **Days 22-23: Advanced Model Evaluation**
    - Generate Confusion Matrices for both Random Forest and XGBoost.
    - Plot the **ROC-AUC** (Receiver Operating Characteristic) and **Precision-Recall** curves to visually compare model performance.
    - Select the final, highest-performing model.
- **Days 24-26: Model Interpretability (SHAP)**
    - Machine learning shouldn't be a "black box." Use **SHAP** (SHapley Additive exPlanations) or standard Feature Importance plots to explain your XGBoost/Random Forest model.

    - *Key questions to answer:* Does the model rely more on depth or magnitude to predict a tsunami? How much do the spatial coordinates influence the prediction?
- **Days 27-30: Documentation & Final Presentation**
    - Clean up your Jupyter Notebook(s) so the code is readable, commented, and reproducible.
    - Write a project summary detailing your clustering methodologies, model metrics, and key geological insights.
    - **Deliverable 2 Complete:** A polished, end-to-end predictive modeling pipeline.
    -

UNIVERSITI MALAYA

UNIVERSITI MALAYA

ACADEMIC SESSION 2025/2026: SEMESTER II

SIT3020

PYTHON FOR DATA SCIENCE

GROUP PROJECT

Machine Learning-Based Prediction of Tsunami Occurrence Using Global Seismic Data

GROUP I

NAME	STUDENT ID
LOO SI JIE	22003392
HAOWEI LIN	S2191801
LEE YU ZHEN	22003337
CHIN MUN CHOON	22003259
MANUSHRI A/P NITHIANANTHAN	23005850
JEEVAKUGAN A/L MURTHY	23006033

Table of Contents

1.0 Abstract **3**
2.0 Introduction **3**
2.1 Objectives 3
2.2 Data Background 3
3.0 Methodology **4**
3.1 Data Cleaning & Preprocessing 4
3.1.1 Data Cleaning 4
3.1.2 Preprocessing Pipeline 4
3.2 Exploratory Data Analysis (EDA) 5
3.3 ML Modelling and Evaluation 6
3.3.1 Experimental Setup and Pipeline Implementation 6
3.3.2 End-to-End Machine Learning Pipeline Structure 6
4.0 Results and Discussion **7**
4.1 Plot Results and Discussion 7
4.2 Final Model Comparison Results 10
4.3 Critical Discussion, Limitations, and Improvements 10
4.4 Confusion Matrix & Learning Curve 11
5.0 Conclusion **12**
6.0 References **13**
7.0 Appendices **14**
7.1 requirements.txt 14
7.2 Coding Script 14

1.0 Abstract

Major earthquake events pose a severe threat to coastal areas due to their capacity to trigger catastrophic tsunamis. This project designs and implements an automated machine learning pipeline to predict the occurrence of a tsunami from a dataset of major earthquakes with magnitudes of 6.5 and above. The dataset includes key features like earthquake magnitude, depth, and station distance. To address significant data quality challenges, including severe right-skewness and deep hypocenter outliers, a preprocessing framework utilizing a RobustScaler transformation to safely handle extreme depth outliers extending up to 600 km was applied, while redundant numerical imputation was omitted due to perfect baseline feature completeness. The data was split into training and testing sets before scaling to eliminate the risk of data leakage. A competitive model tournament testing seven distinct learning algorithms, namely Logistic Regression, K-Nearest Neighbors, Decision Trees, Random Forests, Gradient Boosting, Gaussian Naive Bayes, and Support Vector Machines, was conducted. To account for the dataset's distinct class imbalance, the models were optimized and validated using Stratified 5-Fold Cross-Validation, prioritizing the macro F1-score over standard accuracy to ensure both the minority and majority classes are treated with equal weight. Ultimately, the Random Forest algorithm emerged as the champion model, achieving a peak macro Test F1-score of 62.18% and a Test Accuracy of 71.43%. These results demonstrate that combining robust geometric preprocessing with ensemble classification techniques provides a highly reliable framework for early tsunami risk assessment and disaster mitigation.

2.0 Introduction

2.1 Objectives

The objectives of this report are:

1. To explore the distribution of key earthquake features, including magnitude, depth, significance score, distance-related variables, intensity measures, and geographic location.
2. To examine the relationships between seismic features and tsunami occurrence using descriptive statistics, visualisations, and correlation analysis.
3. To apply and compare machine learning models for binary classification of tsunami occurrence based on earthquake event characteristics, identifying the model with the best predictive performance.

2.2 Data Background

The dataset captures detailed geophysical information for earthquake events from the year 1995 to 2023. It contains 1000 records across 19 features. The data has a mix of continuous numerical variables and categorical features. The modeling pipeline is trained on eight numerical features that capture different physical dimensions of a seismic event. The eight numerical features are magnitude, depth, sig (significance), cdi (community internet intensity), mmi (modified mercalli intensity), nst (number of stations), dmin (minimum distance) and gap (azimuthal gap). The target variable, tsunami, was converted into a binary target variable recording occurrences.

3.0 Methodology

3.1 Data Cleaning & Preprocessing

3.1.1 Data Cleaning

- **Dataset Structural Integrity Verification**

The initial dataset structure contained 1000 row entries across 19 distinct columns with a baseline data type distribution of 6 float64 columns, 5 int64 columns, 7 text-based object columns, and 1 pre-existing category column. A missing value scan detected a total of 1622 blank cells:

- ❖ continent: 716 missing cells
- ❖ alert: 551 missing cells
- ❖ country: 349 missing cells
- ❖ location: 6 missing cells

This audit revealed that the data quality issues were categorical, while all numerical features, such as magnitude, depth, sig, and dmin, were complete with 1000 non-null values.

- **Temporal Feature Engineering & Target Formatting**

The date_time feature was converted from string text into a standardized Python datetime object using pd.to_datetime() to match the format='%d-%m-%Y %H:%M'. The target variable, tsunami, was converted from an integer format to a discrete category data type using .astype('category'), successfully defining the task as a binary classification problem.

- **Dataset Filtering and Feature Selection for Target Modeling**

The dataset was filtered to isolate records from 2013 onwards because prior to that there were no tsunami records. This filtering step established a final target variable distribution consisting of 325 Tsunami events (Class 1) and 128 No Tsunami events (Class 0), representing a 128:325 class imbalance ratio. 8 numeric features were selected for modeling, that is magnitude, depth, sig, cdi, mmi, nst, dmin and gap. This subsetting process finalized the input structure with a feature matrix X shape of (453, 8) and a target vector y shape of (453).

3.1.2 Preprocessing Pipeline

Following the text and structural cleaning, a secure preprocessing framework was built using Scikit-Learn's Pipeline and ColumnTransformer libraries.

- **Data Leakage Prevention:**

Prior to scaling, a stratified 80/20 train-test split partitioned the data into 362 training and 91 independent testing samples. This strategy resulted in a tsunami rate of 71.8% in the training set and 71.4% in the test set. This strict separation ensures that information from the testing set remains completely unseen and does not influence the training framework, thus preventing data leakage.

- **Numerical Scaling and Outlier Mitigation:**

Since the structural audit confirmed that all numerical features were fully complete with zero missing values, numerical imputation steps were omitted to keep the pipeline efficient. However, the feature space contained severe geographic and structural outliers, specifically earthquakes extending over 600 kilometers deep. Following an empirical comparison against StandardScaler and MinMaxScaler, RobustScaler selected for final pipeline execution. Since RobustScaler transforms features using the Interquartile Range (IQR) rather than global variances, it neutralizes the outliers. Under RobustScaler, the

maximum value for depth reached 13.93, while the minimum station distance feature stretched to a maximum of 770.00.

- **Centralized Pipeline Compilation:**

Since the structural audit highlighted some missing values in categorical features, they were excluded from the final pipeline to prevent the introduction of synthetic bias or statistical noise through aggressive imputation. These engineering choices were combined into a master ColumnTransformer that mapped the 8 numerical features through the RobustScaler while setting remainder='drop'. This structured pipeline outputs a clean training shape of (362, 8) and a test shape of (91, 8), ensuring identical statistical transformations are applied across both data splits.

3.2 Exploratory Data Analysis (EDA)

- **Descriptive & Volumetric Analysis:**

The dataset focuses on major earthquakes ranging from magnitudes 6.5 to 9.1 and an average magnitude of 6.94. Most earthquakes were shallow, where 787 events (78.7%) were below 70 km, 155 events (15.5%) were intermediate depth, and 58 events (5.8%) were deep. The tsunami rate was 31.26% for shallow earthquakes, 33.55% for intermediate earthquakes, and 46.55% for deep earthquakes. However, depth alone had only a weak correlation with tsunami occurrence, so it should not be interpreted as a strong single predictor.

- **Target Group & Distribution Variance**

Earthquakes associated with tsunamis had a higher average depth at 87.94 km compared with 68.20 km, a higher average cdi value at 5.01 compared with 2.93, and a higher average dmin value at 2.45 compared with 0.49. In contrast, the average magnitude was almost identical for tsunami and non-tsunami events, which suggests that magnitude by itself is not enough to explain tsunami occurrence in this dataset. The distribution analysis showed that gap, significance, depth, and dmin were heavily right-skewed with skewness values above 2.8, indicating that the dataset contains extreme values and outliers, justifying the use of either Robust scaling or tree-based models over algorithms assuming normal distributions.

- **Spatial & Correlation Mechanics:**

The geographic analysis showed that high recorded major earthquakes occurred around countries located near active tectonic boundaries such as Indonesia (140), Papua New Guinea (81), Chile (42), Vanuatu (36), and Mexico (31). The correlation analysis used point-biserial correlation to examine the relationship between numerical seismic features and the binary tsunami variable. The strongest positive association was found for dmin ($r = 0.4432$), followed by cdi ($r = 0.2922$) and gap ($r = 0.2363$). The number of seismic stations, nst, had a moderate negative association with tsunami occurrence ($r = -0.4260$). Magnitude had almost no relationship with tsunami occurrence ($r = -0.0012$), and its p-value showed that it was not statistically significant. This supports the finding that tsunami occurrence is influenced by a combination of factors rather than earthquake magnitude alone.

- **Chronological Reporting Anomaly:**

The time-series analysis revealed an important data quality issue. There were 547 earthquake records before 2013, but none of them were labelled as tsunami events. In contrast, from 2013 onward, 325 out of 453 records were labelled as tsunami events, giving a tsunami occurrence rate of 71.74%. This sudden change is unlikely to reflect a real transition in earthquake behaviour. It more likely suggests a change in data collection, reporting, or labelling. Therefore, records before 2013 are omitted prior to modelling.

3.3 ML Modelling and Evaluation

This section of the report details the training, optimization, and evaluation of seven machine learning models used to predict tsunami occurrences.

3.3.1 Experimental Setup and Pipeline Implementation

An analysis of the filtered training dataset revealed a reversed target class imbalance, where Class 0 (No Tsunami) with 102 samples (28.2%) and Class 1 (Tsunami) with 260 samples (71.8%). The model revealed an imbalance ratio of 0.4:1. Left unaddressed, models maximize standard accuracy by favoring the majority class, producing catastrophic false negatives for the minority event. To address this, four imbalance configurations were benchmarked using baseline Logistic Regression and KNN setups:

- Baseline (No Handling):

This method achieved a test accuracy of 0.747 and a Macro F1-score of 0.610, serving as the primary benchmark.

- Cost-Sensitive Learning (class_weight='balanced'):

This method adjusts the loss function to penalize minority class errors more heavily, resulting in a Test Accuracy of 0.637 and a lower Macro F1-score of 0.586.

- Random Oversampling:

This method duplicated the minority class examples to balance the training set, which degraded performance to a Test Accuracy of 0.549 and a Macro F1-score of 0.509.

- Synthetic Minority Oversampling Technique (SMOTE):

This method expanded the training set from 362 to 520 synthetic samples, yielding a Test Accuracy of 0.560 and a Macro F1-score of 0.518.

Since the baseline model outperformed all artificial balancing techniques, the empirical design choice was made to proceed without imbalance handling. To guarantee a reliable and balanced evaluation, the Macro F1-Score was selected as the main optimization metric for the final model selection pipeline. The Macro F1-score calculates the precision and recall for both classes independently and averages them with completely equal weight, heavily penalizing models that fail to identify the rarer "No Tsunami" events.

3.3.2 End-to-End Machine Learning Pipeline Structure

To prevent data leakage, a structured scikit-learn pipeline was implemented. The pipeline starts with the numerical features that skipped imputation due to perfect baseline completeness. They were then scaled using the RobustScaler to neutralize the disruptive impact of severe geographic and depth outliers. Categorical columns were dropped cleanly to eliminate statistical noise from extreme missingness. The master preprocessing pipeline was integrated into a GridSearchCV framework paired with a 5-fold Stratified Cross-Validation loop. Stratification ensures each fold retains the 0.4:1 class ratio, stabilizing validation across runs while optimizing directly for the Macro F1-score.

4.0 Results and Discussion

4.1 Plot Results and Discussion

--- Top 5 Countries with Major Earthquakes ---

country	count
Indonesia	140
Papua New Guinea	81
Chile	42
Vanuatu	36
Mexico	31

Table 4.1.1: Top 5 Countries with Major Earthquakes

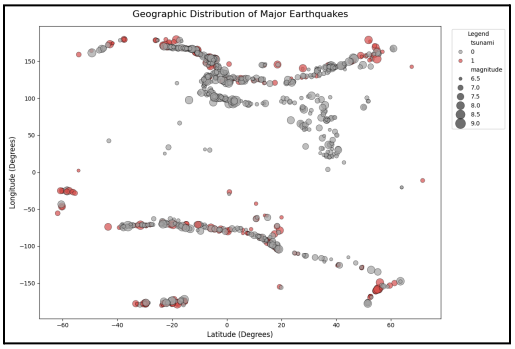


Figure 4.1.1: Geographic Distribution of Major Earthquakes

Table 4.1.1 shows the top five countries with the highest number of major earthquakes in the dataset, led by Indonesia with 140 events, followed by Papua New Guinea (81), Chile (42), Vanuatu (36), and Mexico (31). This concentration reflects their position along active tectonic plate boundaries within the Pacific Ring of Fire, one of the most seismically active regions in the world. Complementing these frequencies, Figure 4.1.1 maps the geographic distribution of these major earthquakes, where point size represents magnitude and color indicates tsunami occurrence. Both earthquake events and tsunami occurrences are tightly clustered within these highly active seismic zones. This spatial concentration confirms that geographic location plays an important role in tsunami risk assessment and can improve the predictive performance of machine learning models.

--- Descriptive Statistics & Coefficient of Variation ---

	count	mean	std	min	max	CV
dmin	1000.0	1.125174	2.073164	0.0	17.654	1.842527
depth	1000.0	74.612541	130.812590	2.7	670.810	1.753225
nst	1000.0	193.918000	239.045858	0.0	934.000	1.232716
gap	1000.0	20.926290	24.415895	0.0	239.000	1.166757
cdi	1000.0	3.605000	3.328972	0.0	9.000	0.923432
sig	1000.0	847.915000	301.802632	650.0	2910.000	0.355935
mmi	1000.0	6.027000	1.433990	1.0	10.000	0.237928
magnitude	1000.0	6.940150	0.438148	6.5	9.100	0.063132

Feature with highest CV: **dmin** (CV = 1.84)

Table 4.1.2: Descriptive Statistics and Coefficient of Variation of Seismic Features

--- Earthquake Count and Percentage by Depth ---

depth_category	Count	Percentage (%)
Shallow	787	78.7
Intermediate	155	15.5
Deep	58	5.8

Table 4.1.3: Distribution of Earthquakes by Depth Category

--- Tsunami Rate by Depth Category ---

	Depth Category	Tsunami Rate (Proportion)	Tsunami Rate (%)
0	Shallow	0.312579	31.26
1	Intermediate	0.335484	33.55
2	Deep	0.465517	46.55

Table 4.1.4: Tsunami Rate by Depth Category

Table 4.1.2 shows the descriptive statistics and coefficient of variation for the seismic features used in this study. Among all variables, depth exhibited the highest variability with a standard deviation of 130.81 and a coefficient of variation of 1.75, indicating substantial dispersion in earthquake depths across the dataset. DMIN also showed considerable variability with a coefficient of variation of 1.84, suggesting large differences in the distance between earthquake events and the nearest seismic station. In contrast, magnitude recorded the lowest coefficient of variation at 0.06, with values ranging only from 6.5 to 9.1 and a mean magnitude of 6.94. These findings indicate that while earthquake magnitudes were relatively consistent, other seismic characteristics varied substantially and may provide additional predictive information for tsunami occurrence. Table 4.1.3 outlines the distribution of earthquakes according to depth category, where the majority were classified as shallow earthquakes, accounting for 787 events or 78.7 percent of the dataset. Intermediate-depth earthquakes represented 15.5 percent with 155 events, while deep earthquakes constituted only 5.8 percent with 58 events. This distribution indicates that most recorded major earthquakes occurred near the Earth surface, which is consistent with global seismic activity patterns where tectonic interactions commonly generate shallow seismic events. Table 4.1.4 shows the tsunami occurrence rate across these different depth categories. Deep earthquakes recorded the highest tsunami occurrence rate at 46.55 percent, followed by intermediate-depth earthquakes at 33.55 percent and shallow earthquakes at 31.26 percent. Although shallow earthquakes were the most common events in the dataset, they exhibited the lowest tsunami occurrence rate. This suggests that earthquake depth may influence tsunami generation and should be considered as an important feature in predictive modelling.

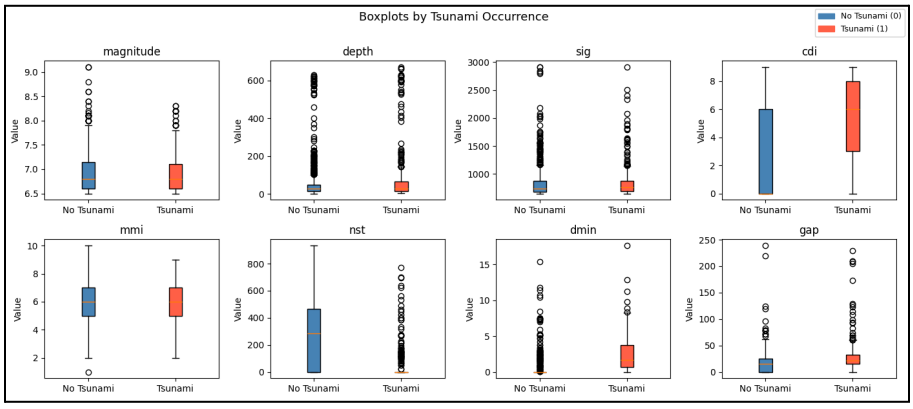


Figure 4.1.2: Comparison of Earthquake Characteristics via Boxplots

Figure 4.1.2 compares the multiple seismic characteristics between tsunami and non-tsunami events using boxplots. From the magnitude boxplot, it is observed that the baseline data is highly concentrated near the

lower threshold of 6.5, while a dense trail of extreme upper outliers extends all the way up to 9.0 across no tsunami class and all the way up to 8.5 for tsunami class. This observation indicates that there is a positive skewness in the magnitude histogram. Similarly, in the gap boxplot, it is shown that the interquartile range is tightly compressed below 50 degrees, while a long string of isolated and high-value outliers stretches up to 250 degrees due to poor station coverage for rare events. This proves that there is a strong right-skewness in the station gap distribution. The nearest station distance distribution is also right-skewed based on the dmin boxplot configuration where the majority of the data is squeezed tightly close to the zero value, while dense vertical columns of upper outliers stretch beyond 15.0 and create the elongated distribution tail. Tsunami events generally exhibit higher values for CDI, DMIN, and GAP compared to non-tsunami events. For example, the median CDI value for tsunami events is noticeably higher, indicating stronger perceived shaking effects, and the DMIN distribution is clearly shifted upward for tsunami events to support the correlation analysis. These distinct differences observed across multiple variables reinforce the need for multivariate machine learning approaches rather than relying on a single predictor.

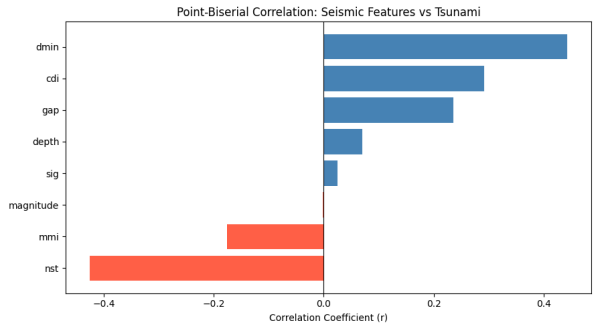


Figure 4.1.3: Point-Biserial Correlation Between Seismic Features and Tsunami Occurrence

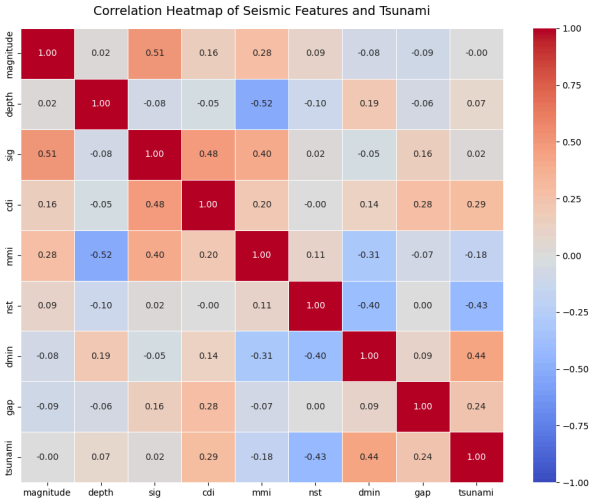


Figure 4.1.4: Correlation Heatmap of Seismic Features and Tsunami Occurrence

Figure 4.1.3 shows the point-biserial correlation coefficients between seismic features and tsunami occurrence, where DMIN recorded the strongest positive correlation with r equals 0.44, followed by CDI at 0.29 and GAP at 0.24. Conversely, NST exhibited the strongest negative correlation with r equals minus 0.43, while MMI displayed a weaker negative relationship at minus 0.18, and magnitude showed almost no correlation. These findings indicate that tsunami generation is influenced by multiple seismic characteristics rather than a single dominant variable. Expanding on this, Figure 4.1.4 shows the full correlation heatmap of all seismic features and tsunami occurrence. The strongest positive correlation among the predictors themselves is observed between magnitude and significance with r equals 0.51, while depth and MMI exhibit the strongest negative relationship at minus 0.52. With respect to tsunami occurrence, the heatmap reinforces that DMIN shows the highest positive correlation followed by CDI and GAP, while no single feature demonstrates an extremely strong relationship on its own. This confirms that tsunami prediction is a complex problem requiring the combined contribution of multiple seismic variables.

4.2 Final Model Comparison Results

Model	Best CV F1 (Macro)	Training F1 (Macro)	Test Accuracy	Test F1 (Macro)	Test Precision (Macro)	Test Recall (Macro)
Random Forest	0.6265	0.7493	0.7143	0.6218	0.6373	0.6154
Logistic Regression	0.6666	0.6701	0.6593	0.6041	0.6013	0.6115
SVM	0.6546	0.6761	0.6264	0.5698	0.5689	0.5769
Naive Bayes	0.6203	0.6431	0.6813	0.5585	0.5775	0.5577
KNN	0.6094	1.0000	0.7253	0.5578	0.6497	0.5654
Gradient Boosting	0.6706	0.9210	0.6923	0.5514	0.5844	0.5538
Decision Tree	0.6061	0.8610	0.6484	0.5204	0.5297	0.5231

Table 4.2.1: Classification Metrics of Machine Learning Models Optimized via Macro F1

4.3 Critical Discussion, Limitations, and Improvements

Model Performance Interpretation

Random Forest is selected as the best overall model for deployment. Although it did not achieve the highest cross-validation score, it demonstrated superior generalization on unseen testing data, securing the peak Test F1 Macro score of 0.6218 and a high Test Accuracy of 0.7143. This indicates that the ensemble's tree-bagging mechanism effectively minimized variance when confronted with out-of-sample data points. On the other hand, several models exhibit clear symptoms of high variance and overfitting to the training set. The K-Nearest Neighbors (KNN) model achieved a perfect Training F1 Macro score of 1.0000 due to its distance-based memorization framework, yet collapsed to a Test F1 Macro score of only 0.5578. Similarly, Gradient Boosting overfit the training set, dropping from a Training F1 Macro score of 0.9210 to a Test F1 Macro score of 0.5514. The Decision Tree followed a similar trajectory, with its Training F1 Macro score of 0.8610 dropping down to 0.5204 on the test set. In contrast, Logistic Regression and Support Vector Machine (SVM) displayed stable learning profiles with minimal generalization gaps. Logistic Regression maintained stability between its Best CV F1 Macro of 0.6666 and its Training F1 Macro score of 0.6701, while delivering a competitive Test F1 Macro score of 0.6041. SVM behaved similarly, with its Best CV F1 Macro score of 0.6546 closely matching its Training F1 Macro score of 0.6761, before securing a Test F1 Macro score of 0.5698. While these linear models generalize reliably, their lower overall test score suggests they struggle to capture the complex and non-linear geometric boundaries of the dataset as effectively as the Random Forest model.

Limitations and Proposed Improvements

The experiment relies on a highly refined dataset of 362 training samples extracted from a total filtered dataset of 453 major earthquake records This limited data scale places an artificial ceiling on complex ensemble models like Gradient Boosting and Random Forests, causing them to overfit because there are not enough unique historical sequences to map. Collecting more diverse and globalised seismic historical records would build a more robust training foundation. Additionally, the pipeline utilizes a default probability threshold of 0.5 to separate classification decisions. In a real-world tsunami warning context, false negatives, the event of predicting "No Tsunami" when a tsunami is happening, carry worse human and economic costs. A crucial next step for this project can involve abandoning static thresholds for a custom cost-benefit matrix. This mathematical adjustment will optimize the classification boundary to aggressively prioritize Test Recall Macro over baseline precision. Furthermore, embedding resampling techniques directly into the cross-validation loops using an imblearn.pipeline.Pipeline would allow synthetic sampling hyper-parameters to tune dynamically alongside model configurations without any risk of data leakage.

4.4 Confusion Matrix & Learning Curve

```
---Confusion Matrix of training set---
[[ 72  30]
 [ 47 213]]
```

Figure 4.4.1: Confusion Matrix of Training Set

```
Confusion Matrix:
[[10 16]
 [10 55]]
```

Figure 4.4.2: Confusion Matrix of Test Set

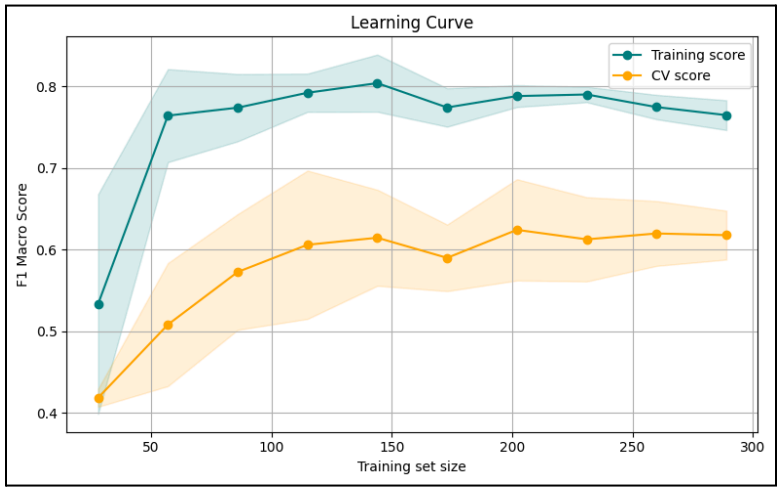


Figure 4.4.3: Predictive Model Learning Curve

Figure 4.4.1 shows the training set confusion matrix highlighting the model initial classification behavior, successfully identifying 72 true negatives and 213 true positives while committing 30 false positives and 47 false negatives. When evaluated against unseen testing data, the test confusion matrix shown in Figure 4.4.2 a matching distribution, securing 10 true negatives and 55 true positives against 16 false positives and 10 false negatives. This out-of-sample breakdown demonstrates that the model avoids collapsing entirely into majority class prediction despite the target imbalance, though the occurrence of 10 false negatives underlines a remaining operational risk of missed tsunami events. This generalization behavior

is further clarified by the learning curve in Figure 4.4.3, which plots the F1 Macro score against an increasing training set size. As the training sample size approaches 300, the training score stabilizes around 0.77 while the cross-validation score plateaus near 0.62. The stable trajectory of both scores at higher sample sizes indicates that the model has successfully converged, though the persistent gap between the training and cross-validation curves confirms a moderate level of variance and mild overfitting caused by the limited scale of available historical seismic records.

5.0 Conclusion

This report analysed a dataset of major earthquake events to explore which earthquake characteristics are related to tsunami occurrence. The exploratory analysis showed that tsunami occurrence cannot be explained by magnitude alone. Although all records were major earthquakes, the average magnitude was almost the same for tsunami and non-tsunami events. Instead, variables such as dmin, cdi, gap, nst, and depth showed stronger differences or associations with the tsunami label. The analysis also found several important data issues. Some administrative and location-based variables had high missing values and could not be reliably imputed. In addition, the absence of tsunami labels before 2013 suggests a possible change in data collection or labelling practices. This is an important limitation because it may affect model training and evaluation. Any predictive model built from this dataset should clearly state whether the full dataset or only the more reliable 2013 onward subset is used. Overall, the results suggest that tsunami prediction requires a combination of seismic, spatial, and temporal features rather than one simple measurement. The EDA helped identify useful predictors, detect outliers and skewed distributions, and highlight limitations that must be considered during modelling. Future work should validate the tsunami labels, use modelling methods that can handle non-linear relationships and outliers, and evaluate performance using metrics that are suitable for imbalanced classification, such as F1-score, precision, and recall.

6.0 References

United States Geological Survey. (2024). Earthquake hazards program. https://earthquake.usgs.gov

U.S. Geological Survey. (n.d.). Earthquake Hazards Program. Retrieved June 13, 2026, from https://www.usgs.gov/programs/earthquake-hazards

National Ocean Service. (2024, June 16). What is a tsunami? National Oceanic and Atmospheric Administration. https://oceanservice.noaa.gov/facts/tsunami.html

McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), Proceedings of the 9th Python in Science Conference (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Waskom, M. L. (2021). seaborn: Statistical data visualization. Journal of Open Source Software, 6(60), 3021. https://doi.org/10.21105/joss.03021

7.0 Appendices

7.1 requirements.txt

```
# SIT3020 Python for Data Science — required packages
# Install all:  pip install -r requirements.txt

# Core data science stack
numpy>=1.23.0
pandas>=1.5.0
matplotlib>=3.6.0
scipy>=1.9.0
seaborn>=0.13.2

# Machine learning
scikit-learn>=1.2.0
mglearn>=0.1.9

# File format support
openpyxl>=3.0.10
pyarrow>=10.0.0

# Jupyter environment
jupyter>=1.0.0
ipykernel>=6.0.0
```

7.2 Coding Script

```
# The other user runs this in their Colab cell
!gdown "11DC9bik8Piyy7tVY-q9-eq59LOXrhsMP"
!gdown "1q0IehwyePJk2zS1ZLUD8gYQS5gdVxZfd"

%matplotlib inline
import warnings
# warnings.filterwarnings('ignore') #To show warnings
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import pickle

print("✅ All libraries loaded!")
```

```
import os

# Change to a new directory using an absolute path
#os.chdir(r"/content")
#print(f"New working directory: {os.getcwd()}")

# ── Read requirements.txt — line by line ──────────────────────────
packages = {}
print("Packages required for SIT3020:")
with open(r"Assignment/requirements.txt", "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if line and not line.startswith("#"):
            print(f"  📦 {line}")
            if ">=" in line:
                pkg, ver = line.split(">=")
                packages[pkg] = ver
            else:
                packages[line] = "0.0.0" # Default minimum version if not specified

for pkg, min_ver in packages.items():
    try:
        mod = __import__(pkg)
        ver = getattr(mod, "__version__", "?")
        ok  = "✅" if ver >= min_ver else "⚠️  version too old"
        print(f"  {ok}  {pkg:12s}  version: {ver}")
    except ImportError:
        print(f"  ❌  {pkg:12s}  NOT FOUND — run: pip install {pkg}")

# ── Load earthquake.csv — the main dataset
df = pd.read_csv(
    "Assignment/earthquake.csv",
    na_values=["N/A","--",""],      # treat these as NaN
)

# Display the first five rows
print(f"Loaded: {df.shape[0]} rows × {df.shape[1]} columns")
print(df.head())

# Drop the identifier columns
# 'title' is dropped here because it is a unique text string for each event,
# not a measurable feature or category.
df.drop(columns=['title'], inplace=True)
```

```
# Handle data type, formatting
df['date_time'] = pd.to_datetime(df['date_time'], format='%d-%m-%Y %H:%M', errors='coerce')
df['year'] = df['date_time'].dt.year

# Confirm conversion
print(f"Date range: {df['year'].min()} to {df['year'].max()}")

# Display the updated shape and confirm remaining columns
print("\n--- Updated Dataset Info ---")
print(f"Updated Shape: {df.shape[0]} rows × {df.shape[1]} columns")
print(f"Remaining Columns:\n{df.columns.tolist()}")

# Check for missing values across all remaining columns
print("Missing values:")
missing = pd.DataFrame({
    "count": df.isnull().sum(),
    "pct(%)": (df.isnull().mean()*100).round(2)
})
# Filter to show only columns that actually have missing values
print(missing[missing["count"]>0])

# 1c. Display the number of earthquake records per country (non-missing only)
print("--- Number of Earthquake Records per Country ---")
# value_counts() automatically excludes missing (NaN) values
country_counts = df['country'].value_counts()
display(country_counts)

# Identify the top five countries with the most recorded major earthquakes
print("\n--- Top 5 Countries with Major Earthquakes ---")
top_5_major_countries = country_counts.head(5)
display(top_5_major_countries)

# 2a. Descriptive Statistics
features = ['magnitude', 'cdi', 'mmi', 'sig', 'nst', 'dmin', 'gap', 'depth']

# Generate standard descriptive stats
desc_stats = df[features].agg(['count', 'mean', 'std', 'min', 'max']).T

# Calculate the Coefficient of Variation (CV = std / mean)
desc_stats['CV'] = desc_stats['std'] / desc_stats['mean']

print("--- Descriptive Statistics & Coefficient of Variation ---")
# Display sorted by CV to easily identify the highest
```

```
display(desc_stats.sort_values(by='CV', ascending=False))

highest_cv_feature = desc_stats['CV'].idxmax()
highest_cv_value = desc_stats['CV'].max()
print(f"\nFeature with highest CV: **{highest_cv_feature}** (CV = {highest_cv_value:.2f})")

# 2b. Compare mean values between tsunami and non-tsunami events
features_to_compare = ['magnitude', 'depth', 'cdi', 'dmin']

print("--- Mean Values by Tsunami Event (0 = No, 1 = Yes) ---")
tsunami_means = df.groupby('tsunami')[features_to_compare].mean()
display(tsunami_means)

# 2c. Classify by depth and calculate tsunami rates
# Re-applying to ensure strict boundaries: Shallow (< 70), Intermediate (70-300), Deep (> 300)
df['depth_category'] = pd.cut(
    df['depth'],
    bins=[0, 70, 300, np.inf],
    labels=['Shallow', 'Intermediate', 'Deep'],
    right=False # Ensures the bin is [0, 70), meaning strictly less than 70
)

# Calculate Count and Percentage
depth_stats = pd.DataFrame({
    'Count': df['depth_category'].value_counts(sort=False),
    'Percentage (%)': (df['depth_category'].value_counts(normalize=True, sort=False) * 100).round(2)
})

print("--- Earthquake Count and Percentage by Depth ---")
display(depth_stats)

# Calculate Tsunami Rate (proportion of tsunami = 1)
# Converting the tsunami column to int temporarily for the mean calculation
tsunami_rate = df.assign(
    tsunami_numeric=df['tsunami'].astype(int)
).groupby('depth_category', observed=False)['tsunami_numeric'].mean().reset_index()

tsunami_rate.columns = ['Depth Category', 'Tsunami Rate (Proportion)']
tsunami_rate['Tsunami Rate (%)'] = (tsunami_rate['Tsunami Rate (Proportion)'] * 100).round(2)

print("\n--- Tsunami Rate by Depth Category ---")
display(tsunami_rate)

# Configurable variables for the plot
```

```
fig_width = 8
fig_height = 5
num_bins = 15
hist_color = 'skyblue'

# Create the figure
fig, ax = plt.subplots(figsize=(fig_width, fig_height))

# Plot the histogram for magnitude
sns.histplot(data=df, x='magnitude', bins=num_bins, color=hist_color, edgecolor='black', kde=False,
ax=ax)

# Add titles and labels
ax.set_title('Distribution of Earthquake Magnitudes (≥ 6.5)', fontsize=14, pad=15)
ax.set_xlabel('Magnitude', fontsize=12)
ax.set_ylabel('Frequency (Count)', fontsize=12)

# Display the plot
plt.tight_layout()
plt.show()

# Configurable variables for the plot
fig_width = 8
fig_height = 6
box_palette = ['#ff9999', '#66b3ff'] # Colors for 0 (No Tsunami) and 1 (Tsunami)

# Create the figure
fig, ax = plt.subplots(figsize=(fig_width, fig_height))

# Create a temporary column for clearer x-axis labels
df['tsunami_label'] = df['tsunami'].map({0: 'No Tsunami (0)', 1: 'Tsunami (1)'})

# Plot the boxplot
sns.boxplot(data=df, x='tsunami_label', y='depth', palette=box_palette, ax=ax)

# Add titles and labels
ax.set_title('Earthquake Depth Distribution by Tsunami Status', fontsize=14, pad=15)
ax.set_xlabel('Tsunami Triggered', fontsize=12)
ax.set_ylabel('Hypocenter Depth (km)', fontsize=12)

# Display the plot
plt.tight_layout()
plt.show()
```

```
# Drop the temporary label column to keep the dataframe clean
df.drop(columns=['tsunami_label'], inplace=True)

# Configurable variables for the plot
fig_width = 12
fig_height = 8
point_alpha = 0.7
min_dot_size = 30
max_dot_size = 300

# Set up the figure
fig, ax = plt.subplots(figsize=(fig_width, fig_height))

# Create the scatter plot with x=latitude and y=longitude.
sns.scatterplot(
    data=df,
    x='latitude',
    y='longitude',
    hue='tsunami',
    size='magnitude',
    sizes=(min_dot_size, max_dot_size),
    alpha=point_alpha,
    palette=['#a6a6a6', '#d9534f'], # Grey for No Tsunami (0), Red for Tsunami (1)
    edgecolor='black',
    linewidth=0.5,
    ax=ax
)

# Add titles and labels
ax.set_title('Geographic Distribution of Major Earthquakes', fontsize=16, pad=15)
ax.set_xlabel('Latitude (Degrees)', fontsize=12)
ax.set_ylabel('Longitude (Degrees)', fontsize=12)

# Move the legend outside the plot so it doesn't cover the data points
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left', title='Legend')

# Display the plot
plt.tight_layout()
plt.show()

# 4a. Use Point-Biserial correlation to compute the correlation between each seismic feature and the
binary target tsunami
# define seismic features and binary target
seismic_features = ['magnitude', 'depth', 'sig', 'cdi', 'mmi',
```

```
                'nst', 'dmin', 'gap']

binary_target = df['tsunami']

results = []

# loop through each features and extract the correlation results (p-value and r)
for feature in seismic_features:
    r, p = stats.pointbiserialr(df[feature], binary_target)
    results.append({
        'Feature'     : feature,
        'r'           : round(r, 4),
        'p-value'     : round(p, 4),
        'Significant' : 'Yes' if p < 0.05 else 'No'
    })

# Sort by absolute r value (strongest first)
results_df = pd.DataFrame(results)
results_df = results_df.sort_values(by='r', key=abs, ascending=False)
results_df = results_df.reset_index(drop=True)
results_df.index += 1

print("Point-Biserial Correlation — Seismic Features vs Tsunami")
print(results_df.to_string())

# 4b. Visualise the Point-Biserial correlation coefficients as a horizontal bar chart
chart_df = results_df.sort_values('r')

colors = ['steelblue' if r > 0 else 'tomato' for r in chart_df['r']]

plt.figure(figsize=(9, 5))
plt.barh(chart_df['Feature'], chart_df['r'], color=colors)
plt.axvline(0, color='black', linewidth=0.8)
plt.xlabel('Correlation Coefficient (r)')
plt.title('Point-Biserial Correlation: Seismic Features vs Tsunami')
plt.tight_layout()
plt.show()

# 4b. Two features with the strongest absolute association with tsunami occurrence.
top2 = results_df.head(2)
print("Two features with the strongest association with tsunami:")
print(f"  1. {top2.iloc[0]['Feature']}  (r = {top2.iloc[0]['r']})")
print(f"  2. {top2.iloc[1]['Feature']}  (r = {top2.iloc[1]['r']})")
```

```
# Check the distribution shape via Skewness
print("--- Skewness of Numerical Features ---")
print("Values > 1 or < -1 indicate a highly skewed distribution.")
numeric_cols = ['magnitude', 'depth', 'sig', 'cdi', 'mmi', 'nst', 'dmin', 'gap']
skewness = df[numeric_cols].skew().sort_values(ascending=False)
display(pd.DataFrame({'Skewness': skewness}))

# Create a figure and a set of subplots for 'gap'
fig_gap, ax_gap = plt.subplots(1, 1, figsize=(6, 5))

# Plot histogram for 'gap'
df['gap'].hist(ax=ax_gap, bins=30, edgecolor='black')
ax_gap.set_title('Distribution of gap')
ax_gap.set_xlabel('gap values')
ax_gap.set_ylabel('Frequency')

plt.tight_layout()
plt.show()

# Create a figure and a set of subplots for 'dmin'
fig_dmin, ax_dmin = plt.subplots(1, 1, figsize=(6, 5))

# Plot histogram for 'dmin'
df['dmin'].hist(ax=ax_dmin, bins=30, edgecolor='black')
ax_dmin.set_title('Distribution of dmin')
ax_dmin.set_xlabel('dmin values')
ax_dmin.set_ylabel('Frequency')

plt.tight_layout()
plt.show()

fig_magnitude, ax_magnitude = plt.subplots(1, 1, figsize=(6, 5))

df['magnitude'].hist(ax=ax_magnitude, bins=20, edgecolor='black')
ax_magnitude.set_title('Distribution of Magnitude')
ax_magnitude.set_xlabel('Magnitude')
ax_magnitude.set_ylabel('Frequency')

plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 8))
# Compute correlation matrix for numeric columns
corr_matrix = df[numeric_cols + ['tsunami']].corr()
```

```
# Plot the heatmap
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f",
            linewidths=0.5, vmin=-1, vmax=1)
plt.title('Correlation Heatmap of Seismic Features and Tsunami', fontsize=14, pad=15)
plt.tight_layout()
plt.show()

# Boxplots for numeric features — split by tsunami (0 = No, 1 = Yes)
fig, axes = plt.subplots(2, 4, figsize=(14, 6))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    group0 = df[df['tsunami'] == 0][col].dropna()
    group1 = df[df['tsunami'] == 1][col].dropna()

    bp = axes[i].boxplot([group0, group1],
                         patch_artist=True,
                         tick_labels=['No Tsunami', 'Tsunami'])

    bp['boxes'][0].set_facecolor('steelblue')
    bp['boxes'][1].set_facecolor('tomato')

    axes[i].set_title(col)
    axes[i].set_ylabel('Value')

# Shared legend
from matplotlib.patches import Patch
legend = [Patch(color='steelblue', label='No Tsunami (0)'),
          Patch(color='tomato',    label='Tsunami (1)')]
fig.legend(handles=legend, loc='upper right', fontsize=9)

plt.suptitle('Boxplots by Tsunami Occurrence', fontsize=13, y=1.01)
plt.tight_layout()
plt.show()

# Calculate annual earthquake counts
earthquake_counts = df.set_index('date_time').resample('YE').size().reset_index(name='count')
earthquake_counts['event_type'] = 'Earthquakes'
earthquake_counts['year'] = earthquake_counts['date_time'].dt.year

# Calculate annual tsunami counts
tsunami_events = df[df['tsunami'] == 1]
tsunami_counts = tsunami_events.set_index('date_time').resample('YE').size().reset_index(name='count')
```

```
tsunami_counts['event_type'] = 'Tsunamis'
tsunami_counts['year'] = tsunami_counts['date_time'].dt.year

# Combine the dataframes
combined_counts = pd.concat([earthquake_counts[['year', 'event_type', 'count']], tsunami_counts[['year',
'event_type', 'count']]])

# Create the line chart
plt.figure(figsize=(12, 7))
sns.lineplot(x='year', y='count', hue='event_type', data=combined_counts)
plt.title('Annual Count of Earthquakes and Tsunami Events Over Time')
plt.xlabel('Year')
plt.ylabel('Number of Events')
plt.legend(title='Event Type')
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

# Define the periods
df['period'] = df['year'].apply(lambda x: 'Before 2013' if x < 2013 else '2013 and Forward')

# Group by period and calculate total events and tsunami events
period_summary = df.groupby('period').agg(
    total_events=('year', 'count'),
    tsunami_events=('tsunami', lambda x: (x == 1).sum())
).reset_index()

# Calculate the probability of tsunami occurrence
period_summary['tsunami_probability'] = period_summary['tsunami_events'] /
period_summary['total_events']

print("Comparison of Tsunami Occurrence Probability:")
display(period_summary.round(4))

# Visualize the comparison
plt.figure(figsize=(8, 6))
sns.barplot(x='period', y='tsunami_probability', data=period_summary, palette='viridis')
plt.title('Tsunami Occurrence Probability: Before 2013 vs. 2013 and Forward')
plt.xlabel('Period')
plt.ylabel('Probability of Tsunami Occurrence')
plt.ylim(0, period_summary['tsunami_probability'].max() * 1.1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.preprocessing import RobustScaler, OrdinalEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, f1_score,
    classification_report, confusion_matrix,
)
from imblearn.over_sampling import SMOTE, RandomOverSampler


#Filter Data Based on EDA Insights
df_filtered = df[df['year'] >= 2013].copy()

print("\nTarget distribution (tsunami):")
print(df_filtered['tsunami'].value_counts())
print(
    f"Class imbalance ratio: "
    f"{df_filtered['tsunami'].value_counts()[0]} (No Tsunami) : "
    f"{df_filtered['tsunami'].value_counts()[1]} (Tsunami)"
)

# Base numeric features (no engineered cols yet — those come in Step 2)
numeric_features = ['magnitude', 'depth', 'sig', 'cdi', 'mmi', 'nst', 'dmin', 'gap',]

X = df_filtered[numeric_features].copy()
y = df_filtered['tsunami'].copy()

print(f"\nFeatures selected : {len(numeric_features)}")
print(f"  Numeric features          : {numeric_features}")
print(f"\nX shape: {X.shape}    y shape: {y.shape}")

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size    = 0.2,
    stratify     = y,       # keeps same class % in train and test, note that the data is not time series trend
    random_state = 42
```

```
)

print(f"Training set   : {X_train.shape[0]} rows")
print(f"Test set       : {X_test.shape[0]} rows")
print(f"\nClass proportions after stratified split:")
print(f"  Train — tsunami rate : {y_train.astype(int).mean():.3f}")
print(f"  Test  — tsunami rate : {y_test.astype(int).mean():.3f}")
print()



from sklearn.preprocessing import StandardScaler, MinMaxScaler

X_num_train = X_train[numeric_features].copy()
X_num_test  = X_test[numeric_features].copy()

ss  = StandardScaler()
mms = MinMaxScaler(feature_range=(0, 1))
rs  = RobustScaler()

X_ss_train  = ss.fit_transform(X_num_train)
X_mm_train  = mms.fit_transform(X_num_train)
X_rs_train  = rs.fit_transform(X_num_train)

print("--- Comparison of Scalers Across All Numeric Features ---")

for feature_name in numeric_features:
    print(f"\nFeature: '{feature_name}'")
    print(f"{'Scaler':<20} {'min':>8} {'max':>8} {'mean':>8} {'std':>8}")
    print("-" * 55)

    # Get the column index for the current feature
    col_idx = features.index(feature_name)

    for name, arr in [("StandardScaler", X_ss_train),
                      ("MinMaxScaler",   X_mm_train),
                      ("RobustScaler",   X_rs_train)]:
        # Extract the specific feature's column from the scaled matrix
        col = arr[:, col_idx]
        print(f"{name:<20} {col.min():>8.2f} {col.max():>8.2f} "
              f"{col.mean():>8.2f} {col.std():>8.2f}")

print("\nVerdict: RobustScaler chosen — handles right-skewed features and extreme outliers safely.")
```

```
counts = y_train.value_counts()
total  = len(y_train)
print(f"\nClass 0 (no tsunami)  : {counts[0]:>4}  ({counts[0]/total*100:.1f}%)")
print(f"Class 1 (tsunami)     : {counts[1]:>4}  ({counts[1]/total*100:.1f}%)")
print(f"Imbalance ratio       : {counts[0]/counts[1]:.1f} : 1\n")

# Scaled arrays for this section (rs already fit on train)
X_tr_sc = rs.fit_transform(X_train[numeric_features])
X_te_sc = rs.transform(X_test[numeric_features])

results = {}

# --- Baseline ---
# We use standard Logistic Regression without touching weights
lr_base = LogisticRegression(max_iter=1000, random_state=42)
lr_base.fit(X_tr_sc, y_train)
y_pred_base = lr_base.predict(X_te_sc)
results['Baseline (no handling)'] = {
    'acc': accuracy_score(y_test, y_pred_base),
    'f1':  f1_score(y_test, y_pred_base, average='macro', zero_division=0),
}

# --- class_weight='balanced' (Logistic Regression) ---
lr_bal = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
lr_bal.fit(X_tr_sc, y_train)
y_pred_bal = lr_bal.predict(X_te_sc)
results['class_weight=balanced'] = {
    'acc': accuracy_score(y_test, y_pred_bal),
    'f1':  f1_score(y_test, y_pred_bal, average='macro', zero_division=0),
}

# --- Random Oversampling ---
ros = RandomOverSampler(random_state=42)
X_ros, y_ros = ros.fit_resample(X_tr_sc, y_train)
knn_ros = KNeighborsClassifier(n_neighbors=5)
knn_ros.fit(X_ros, y_ros)
y_pred_ros = knn_ros.predict(X_te_sc)
results['Random oversampling'] = {
    'acc': accuracy_score(y_test, y_pred_ros),
    'f1':  f1_score(y_test, y_pred_ros, average='macro', zero_division=0),
}
print(f"Random oversampling: class 1 grew from {counts[1]} → {(y_ros==1).sum()}")

# --- SMOTE ---
```

```
k_smote = min(5, counts[1] - 1)
smote   = SMOTE(k_neighbors=k_smote, random_state=42)
X_smote, y_smote = smote.fit_resample(X_tr_sc, y_train)
knn_smote = KNeighborsClassifier(n_neighbors=5)
knn_smote.fit(X_smote, y_smote)
y_pred_smote = knn_smote.predict(X_te_sc)
results['SMOTE'] = {
   'acc': accuracy_score(y_test, y_pred_smote),
   'f1':  f1_score(y_test, y_pred_smote, average='macro', zero_division=0),
}
print(f"SMOTE: {len(y_train)} samples → {len(y_smote)} synthetic samples")

print(f"\n{'Method':<30} {'Accuracy':>10} {'F1 Score':>10}")
print("-" * 55)
for method, scores in results.items():
   print(f"{method:<30} {scores['acc']:>10.3f} {scores['f1']:>10.3f}")

print(
   "\nKey insight: Macro F1 score  is used because it treats the minority class (No Tsunami) and majority class (Tsunami) exactly equally, it prevents the majority class from inflating overall score as both class accuracy is important in real life. "
   "\nThe baseline model outperforms all balancing techniques. Empirical testing revealed that synthetic oversampling (SMOTE) and class weighting degraded the Macro F1-score from 0.610 to ~0.51-0.58. "
   "\nBecause the baseline model generalised better without artificial manipulation, we opted to proceed with no imbalance handling."
)

# --- Sub-pipeline: numeric columns ---
numeric_pipe = Pipeline([
   ('scaler',  RobustScaler()),                  # robust to depth outliers
])

# --- ColumnTransformer ---
transformers = [('numeric', numeric_pipe, numeric_features)]

preprocessor = ColumnTransformer(transformers=transformers, remainder='drop')

# --- Full pipeline: preprocessor + classifier (BASELINE, NO HANDLING) ---
full_pipeline = Pipeline([
   ('preprocessor', preprocessor),
   ('classifier',  LogisticRegression(max_iter=1000, random_state=42)),
])

print("✅ Final pipeline built:")
```

```
print("   preprocessor (scale) → Logistic Regression (Baseline)\n")

# --- Train ---
full_pipeline.fit(X_train, y_train)

# --- Evaluate on held-out test set ---
y_pred_pipe = full_pipeline.predict(X_test)

acc_pipe = accuracy_score(y_test, y_pred_pipe)
f1_pipe  = f1_score(y_test, y_pred_pipe, average='macro', zero_division=0)

print(f"Pipeline results on test set:")
print(f"  Accuracy : {acc_pipe:.3f}")
print(f"  F1 Score : {f1_pipe:.3f}")
print("\nClassification report:")
print(classification_report(
    y_test, y_pred_pipe,
    target_names=['No Tsunami', 'Tsunami'],
    zero_division=0,
))

cm = confusion_matrix(y_test, y_pred_pipe)
print("Confusion matrix:")
print(f"  True Negative  (correctly predicted NO Tsunami) : {cm[0][0]}")
print(f"  False Positive (false alarm)                    : {cm[0][1]}")
print(f"  False Negative (MISSED a real Tsunami) ⚠️        : {cm[1][0]}")
print(f"  True Positive  (correctly predicted Tsunami)    : {cm[1][1]}")

skf    = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_acc = cross_val_score(full_pipeline, X_train, y_train, cv=skf, scoring='accuracy')
cv_f1  = cross_val_score(full_pipeline, X_train, y_train, cv=skf, scoring='f1_macro')

print(f"\nAccuracy per fold : {[round(v, 3) for v in cv_acc]}")
print(f"Accuracy mean     : {cv_acc.mean():.3f} ± {cv_acc.std():.3f}")
print(f"\nF1 per fold       : {[round(v, 3) for v in cv_f1]}")
print(f"F1 mean           : {cv_f1.mean():.3f} ± {cv_f1.std():.3f}")
print(
    "\nInterpretation: The model demonstrated reasonable stability, with test set performance (F1 = 0.610) falling within the expected variance of the 5-fold cross-validation scores (F1 = 0.570 ± 0.058)."
    "\nSame goes with the accuracy. The overall alignment between CV and test metrics indicates the model generalized without severe overfitting."
)

# Preprocessor-Only Pipeline (For Inspection Only)
```

```
preprocess_only = Pipeline([('preprocessor', preprocessor)])

X_train_preprocessed = preprocess_only.fit_transform(X_train)
X_test_preprocessed  = preprocess_only.transform(X_test)

print(f"X_train_preprocessed shape : {X_train_preprocessed.shape}")
print(f"X_test_preprocessed shape  : {X_test_preprocessed.shape}")


# Fit all models
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    precision_score,
    recall_score,
    classification_report,
    confusion_matrix
)


class ModelSelector:

    def __init__(self, x_train, x_test, y_train, y_test, features=None):
        self.x_train = x_train
        self.x_test = x_test
        self.y_train = y_train
        self.y_test = y_test
        self.features = features

        self.cv = StratifiedKFold(
            n_splits=5,
            shuffle=True,
            random_state=42
```

```
    )

    self.scoring_metric = "f1_macro"
    self.best_models = {}
    self.results = []

    self.best_model_name = None
    self.best_model = None
    self.results_df = None

def define_models(self):

    self.models = {

        "Logistic Regression": {
            "estimator": Pipeline([
                ("scaler", RobustScaler()),
                ("model", LogisticRegression(
                    max_iter=1000,
                    class_weight="balanced",
                    solver="lbfgs",
                    random_state=42
                ))
            ]),
            "param_grid": {
                "model__C": [0.001, 0.01, 0.1, 1, 10, 100]
            }
        },

        "KNN": {
            "estimator": Pipeline([
                ("scaler", RobustScaler()),
                ("model", KNeighborsClassifier())
            ]),
            "param_grid": {
                "model__n_neighbors": [3, 5, 7, 9, 11, 15],
                "model__weights": ["uniform", "distance"]
            }
        },

        "Decision Tree": {
            "estimator": DecisionTreeClassifier(
                random_state=42
            ),
```

```
    "param_grid": {
        "criterion": ["gini", "entropy"],
        "max_depth": [None, 3, 5],
        "min_samples_split": [10, 20],
        "min_samples_leaf": [5, 10]
    }
},

"Random Forest": {
    "estimator": RandomForestClassifier(
        random_state=42,
        n_jobs=-1,
        bootstrap=True,
        oob_score=True
    ),
    "param_grid": {
        "n_estimators": [100, 200],
        "criterion": ["gini", "entropy"],
        "max_depth": [2, 3],
        "min_samples_split": [10, 20],
        "min_samples_leaf": [5, 10],
        "max_features": ["sqrt", "log2"],
        "class_weight": [None, "balanced"]
    }
},

"Gradient Boosting": {
    "estimator": GradientBoostingClassifier(
        random_state=42
    ),
    "param_grid": {
        "n_estimators": [100, 200],
        "learning_rate": [0.01, 0.1, 0.2],
        "max_depth": [2, 3],
        "subsample": [0.8, 1.0]
    }
},

"Naive Bayes": {
    "estimator": GaussianNB(),
    "param_grid": {
        "var_smoothing": [1e-12, 1e-11, 1e-10, 1e-9, 1e-8, 1e-7]
    }
},
```

```
        "SVM": {
            "estimator": Pipeline([
                ("scaler", RobustScaler()),
                ("model", SVC(
                    random_state=42
                ))
            ]),
            "param_grid": {
                "model__C": [0.1, 1, 10, 100],
                "model__kernel": ["linear", "rbf"],
                "model__gamma": ["scale", "auto"],
                "model__class_weight": [None, "balanced"]
            }
        }
    }

def train_one_model(self, model_name, model_info):

    print("\n" + "=" * 70)
    print(f"Training model: {model_name}")
    print("=" * 70)

    grid = GridSearchCV(
        estimator=model_info["estimator"],
        param_grid=model_info["param_grid"],
        cv=self.cv,
        scoring=self.scoring_metric,
        n_jobs=-1,
        error_score="raise"
    )

    grid.fit(self.x_train, self.y_train)

    y_pred = grid.predict(self.x_test)

    train_score = grid.score(self.x_train, self.y_train)
    test_accuracy = accuracy_score(self.y_test, y_pred)
    test_f1 = f1_score(self.y_test, y_pred, average="macro")
    test_precision = precision_score(
        self.y_test,
        y_pred,
        average="macro",
        zero_division=0
```

```
        )
        test_recall = recall_score(
            self.y_test,
            y_pred,
            average="macro",
            zero_division=0
        )

        print("Best params:", grid.best_params_)
        print("Best CV score:", grid.best_score_)
        print("Training F1 macro:", train_score)
        print("Test accuracy:", test_accuracy)
        print("Test F1 macro:", test_f1)

        print("\nConfusion Matrix:")
        print(confusion_matrix(self.y_test, y_pred))

        print("\nClassification Report:")
        print(classification_report(self.y_test, y_pred, zero_division=0))

        self.best_models[model_name] = grid.best_estimator_

        self.results.append({
            "Model": model_name,
            "Best CV F1 Macro": grid.best_score_,
            "Training F1 Macro": train_score,
            "Test Accuracy": test_accuracy,
            "Test F1 Macro": test_f1,
            "Test Precision Macro": test_precision,
            "Test Recall Macro": test_recall,
            "Best Params": grid.best_params_
        })

    def train_all_models(self):

        self.define_models()

        for model_name, model_info in self.models.items():
            self.train_one_model(model_name, model_info)

    def compare_models(self):

        self.results_df = pd.DataFrame(self.results)
```

```
        self.results_df = self.results_df.sort_values(
            by="Test F1 Macro",
            ascending=False
        )

        print("\n" + "=" * 70)
        print("FINAL MODEL COMPARISON")
        print("=" * 70)
        print(self.results_df)

        self.best_model_name = self.results_df.iloc[0]["Model"]
        self.best_model = self.best_models[self.best_model_name]

        print("\nBest Overall Model:", self.best_model_name)
        print("Best Model:")
        print(self.best_model)

        return self.results_df

    def save_results(self, filename="model_comparison_results.csv"):

        if self.results_df is not None:
            self.results_df.to_csv(filename, index=False)
            print(f"\nModel comparison results saved to: {filename}")
        else:
            print("\nNo results found. Please run compare_models() first.")

    def save_best_model(self, filename="best_classification_model.pkl"):

        if self.best_model is not None:
            with open(filename, "wb") as f:
                pickle.dump(self.best_model, f)
                print(f"\nBest model saved to: {filename}")
        else:
            print("\nNo best model found. Please run compare_models() first.")

# Select the best model
Selector = ModelSelector(
    x_test= X_test_preprocessed,
    x_train= X_train_preprocessed,
    y_train= y_train,
    y_test = y_test,
    features = features
)
```

```
Selector.train_all_models()

results_df = Selector.compare_models()
Selector.save_best_model("best_model.pkl")

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

best_rf = RandomForestClassifier(
    class_weight='balanced',
    max_depth=3,
    min_samples_leaf=10,
    min_samples_split=10,
    n_jobs=-1,
    random_state=42
)

# 1. Explicitly fit the model to your training data
best_rf.fit(X_train_preprocessed, y_train)

# 2. Predict on the training data
y_train_pred = best_rf.predict(X_train_preprocessed)

# 3. Generate and plot the training confusion matrix
print("---Confusion Matrix of training set---")
print(confusion_matrix(y_train, y_train_pred))


from sklearn.model_selection import learning_curve, validation_curve

# ============================================
# Plot 1: Learning Curve
# ============================================
train_sizes, train_scores, cv_scores = learning_curve(
    estimator=best_rf,
    X=X_train_preprocessed,
    y=y_train,
    train_sizes=np.linspace(0.1, 1.0, 10),
    cv=StratifiedKFold(
        n_splits=5,
        shuffle=True,
        random_state=42
    ),
    scoring='f1_macro',
```

```
    n_jobs=-1,
    shuffle=True,
    random_state=42
)

train_mean = train_scores.mean(axis=1)
cv_mean    = cv_scores.mean(axis=1)
train_std  = train_scores.std(axis=1)
cv_std     = cv_scores.std(axis=1)

plt.figure(figsize=(8, 5))
plt.plot(train_sizes, train_mean, 'o-', color='teal', label='Training score')
plt.fill_between(train_sizes, train_mean-train_std, train_mean+train_std, alpha=0.15, color='teal')
plt.plot(train_sizes, cv_mean, 'o-', color='orange', label='CV score')
plt.fill_between(train_sizes, cv_mean-cv_std, cv_mean+cv_std, alpha=0.15, color='orange')
plt.xlabel('Training set size'); plt.ylabel('F1 Macro Score')
plt.title('Learning Curve'); plt.legend(); plt.grid(True); plt.tight_layout(); plt.show()
```

Here is the updated model comparison table reflecting your new results. Notice that Gradient Boosting's test performance dropped with the new hyperparameters (`learning_rate`: 0.2, `max_depth`: 2, `n_estimators`: 100), moving it down the rankings, while Random Forest securely maintains its spot as the best overall model.

Model	Best CV F1 Macro	Training F1 Macro	Test Accuracy	Test F1 Macro	Test Precision Macro	Test Recall Macro	Best Parameters
Random Forest (Best Overall)	0.6265	0.7493	0.7143	**0.6218**	0.6373	0.6154	{'class_weight': 'balanced', 'criterion': 'gini', 'max_depth': 3, 'max_features': 'sqrt', 'min_samples_leaf': 10, 'min_samples_split': 10, 'n_estimators': 100}
Logistic Regression	0.6666	0.6701	0.6593	**0.6041**	0.6013	0.6115	{'model__C': 0.1}
SVM	0.6546	0.6761	0.6264	**0.5698**	0.5689	0.5769	{'model__C': 1, 'model__class_weight': 'balanced', 'model__gamma': 'scale', 'model__kernel': 'linear'}

Naive Bayes	0.6203	0.6431	0.6813	**0.5585**	0.5775	0.5577	{'var_smoothing': 1e-12}
KNN	0.6094	1.0000	0.7253	**0.5578**	0.6497	0.5654	{'model__n_neighbors': 9, 'model__weights': 'distance'}
Gradient Boosting	0.6706	0.9210	0.6923	**0.5514**	0.5844	0.5538	{'learning_rate': 0.2, 'max_depth': 2, 'n_estimators': 100, 'subsample': 1.0}
Decision Tree	0.6061	0.8610	0.6484	**0.5204**	0.5297	0.5231	{'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 10}

The table is sorted in descending order by **Test F1 Macro**.

Here is the reinterpretation of the model performance, strictly focused on the Train F1, CV F1, and Test F1 scores, organized within your requested framework:

The Optimal Model

- **Random Forest (The Balanced Performer):** * **Scores:** Train F1: 0.7493 | CV F1: 0.6265 | Test F1: 0.6218
    - **Insight:** Selected as the best model because it achieved the highest Test F1 score on unseen data. Notably, its CV F1 score was an incredibly accurate predictor of its true performance (a gap of only ~0.005), proving that the ensemble mechanism successfully learned underlying patterns without over-memorizing the training set.

Overfitting Diagnosis

- **K-Nearest Neighbors (The Memorizer):**
    - **Scores:** Train F1: 1.0000 | CV F1: 0.6094 | Test F1: 0.5578
    - **Insight:** Exhibits textbook severe variance. It achieved a perfect training score by memorizing exact data points, but collapsed on unseen data. The CV F1 underestimated the severity of this drop-off.
- **Gradient Boosting (The Deceptive CV):**
    - **Scores:** Train F1: 0.9210 | CV F1: 0.6706 | Test F1: 0.5514
    - **Insight:** Showcased the highest CV F1 score of all models, making it look like a top contender. However, the massive gap between its Train F1 and its ultimate Test F1 reveals it aggressively overfit the training data and failed to generalize.
- **Decision Tree (The Poor Generalizer):**
    - **Scores:** Train F1: 0.8610 | CV F1: 0.6061 | Test F1: 0.5204
    - **Insight:** Follows a similar overfitting pattern. It learned the training data well but secured the absolute lowest Test F1 score across the entire board.

High Generalization

- **Logistic Regression (The Stable Runner-Up):**
    - **Scores:** Train F1: 0.6701 | CV F1: 0.6666 | Test F1: 0.6041
    - **Insight:** Displays highly consistent learning metrics. The exceptionally tight grouping between the Train, CV, and Test F1 scores shows this model is highly resistant to overfitting, securing the second-best performance overall.
- **Support Vector Machine (The Consistent Baseline):**
    - **Scores:** Train F1: 0.6761 | CV F1: 0.6546 | Test F1: 0.5698
    - **Insight:** Much like Logistic Regression, SVM shows a very stable profile with minimal gaps between training, cross-validation, and testing. While these linear models generalize reliably and do not overfit, their lower Test

F1 ceilings indicate they cannot capture the dataset's complex boundaries quite as well as the Random Forest.